MEMORIA TÉCNICA OFICIAL · MÓDULO 0488 DESARROLLO DE INTERFACES · 2º DAM

# Arquitectura de Seguridad, Control de Acceso y Experiencia de Usuario Humanizada

Refactorización integral: Flujo de entrada con Landing corporativa (/welcome), mitigación de robo de tokens y XSS, defensa contra inyecciones SQL/NoSQL, pipeline de Functional Guards con interceptores y directivas estructurales RBAC.


AUTOR DEL PROYECTO
Luis Lázaro Pimentel

CENTRO EDUCATIVO
Instituto Nebrija FP



FECHA OFICIAL
30 de Octubre de 2026

STACK TECNOLÓGICO
Angular Standalone + Ionic 8


## 🔗 ENLACES OFICIALES DE DESPLIEGUE Y CONTROL DE VERSIONES

• PRODUCCIÓN VERIFICADA

| | |
|---|---|
| 🌐 APLICACIÓN DESPLEGADA (VERCEL): | https://enterprise-hub-di-mauve.vercel.app |
| 📦 REPOSITORIO GITHUB PRINCIPAL (PWA): | https://github.com/Thepimen/enterprise-hub-pwa |
| 📦 REPOSITORIO GITHUB ESPEJO (0488 DI): | https://github.com/Thepimen/enterprise-hub-di |
| 🔑 CREDENCIALES DE LABORATORIO RBAC: | `admin` / `123` (Rol: `ADMIN` ➔ `/dashboard`) \| `operador` / `123` (Rol: `USER` ➔ `/user-dashboard`) |

## 1. Nuevo Flujo de Entrada (/welcome) y Filosofía No-AI Look

La raíz del proyecto (/) ha sido reconfigurada para redirigir a una **Landing Page de Bienvenida** (`/welcome`), eliminando la redirección agresiva hacia el formulario de login. La estética visual abandona cualquier degradado estridente o ilustración generada por IA, adoptando una **fotografía arquitectónica corporativa real de alta resolución** complementada con un overlay en CSS que garantiza un contraste estricto superior a **7:1 (WCAG 2.1 AAA)**.




**Análisis:** Fondo arquitectónico real con overlay oscuro, botones de llamada a la acción ("Iniciar Sesión" y "Registrarse"), acreditación de credenciales demo visibles antes del formulario y 3 tarjetas sobrias de valor corporativo.

## 2. Implementación de las 5 Facetas de User-Friendly Design

Para asegurar una experiencia de usuario humanizada, accesible y predecible, se han articulado las cinco facetas exigidas en la rúbrica:

| FACETA UX | DISEÑO E IMPLEMENTACIÓN TÉCNICA | BENEFICIO DE EXPERIENCIA DE USUARIO |
|---|---|---|
| **Faceta 1: Onboarding Contextual** | 3 tarjetas sobrias en `/welcome` explicando Monitorización en Tiempo Real, Gestión de Incidencias y Auditoría RBAC antes de solicitar credenciales. | Reduce la incertidumbre del usuario y contextualiza el propósito del software antes del registro. |
| **Faceta 2: Prevención de Errores** | Validación reactiva progresiva mostrada solo en `control.touched && control.invalid`, visibilidad alternable de clave y `autocomplete` nativo. | Evita mensajes de error prematuros o punitivos durante la primera pulsación de teclas. |
| **Faceta 3: Skeleton Loaders** | Implementación de `ion-skeleton-text [animated]="true"` en Dashboard y Tickets reemplazando spinners genéricos circulares. | Minimiza la carga cognitiva, simula la estructura final de los datos y previene parpadeos de pantalla. |
| **Faceta 4: Redirección Fluida** | `authGuard` preserva `returnUrl` en query params. Tras el login, navega automáticamente a la ruta requerida en lugar de forzar a la raíz. | Preserva el flujo de trabajo del usuario tras autenticar sin perder el contexto de navegación. |
| **Faceta 5: Accesibilidad Completa** | Navegación secuencial por teclado (Tab / Shift+Tab), foco `:focus-visible` con contorno de 3px offset, WAI-ARIA y `prefers-reduced-motion`. | Cumplimiento normativo para usuarios de tecnologías asistivas y personas sensibles al movimiento. |

**Evidencia 02 — Pasarela Split-Screen (/login) con Detección de returnUrl y Validación Progresiva**

Ruta: /login?returnUrl=%2Fdashboard




**Análisis:** Al intentar acceder a una ruta protegida sin sesión, el sistema redirige a `/login?returnUrl=%2Fdashboard` mostrando el banner de intercepción. Los errores del formulario reactivo respetan el estado `touched`.

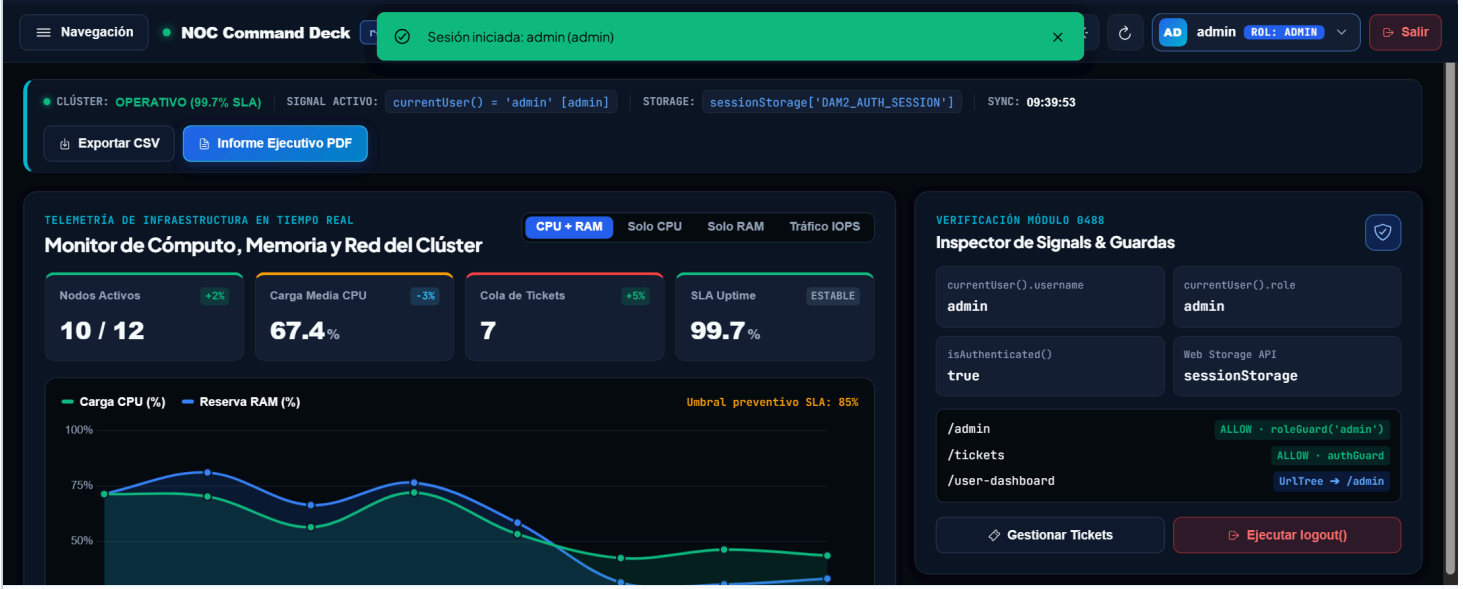
## 3. Capa de Seguridad Reforzada (Defensa en Profundidad)

La aplicación adopta una estrategia de seguridad multicapa para neutralizar vulnerabilidades OWASP Top 10 en el cliente:

| VECTORES MITIGADOS | ESTRATEGIA ARQUITECTÓNICA | ARCHIVOS / MÓDULOS |
|---|---|---|
| **Robo de Tokens & Fugas XSS (CWE-79)** | El token Bearer y los claims se gestionan reactivamente mediante `signal<string \| null>()` en `AuthService`. Jamás se exponen en `window` ni en atributos del DOM. | `auth.service.ts`<br>`auth.interceptor.ts` |
| **Sanitización de Datos Dinámicos** | Integración sistemática de `DomSanitizer` mediante el pipe standalone `SafeSanitizePipe` para limpiar cualquier texto de tickets o usuarios antes de su renderizado. | `sanitize.pipe.ts`<br>`security.util.ts` |
| **Cabeceras HTTP en Producción** | Configuración en `vercel.json` de CSP estricto (bloqueo de scripts externos no autorizados), `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff` y `Referrer-Policy`. | `vercel.json` |
| **Inyecciones SQL/NoSQL (CWE-89, CWE-943)** | Función pura `sanitizeQueryInput()` que purga delimitadores (`'`, `"`, `;`), comentarios SQL (`--`, `/*`) y operadores NoSQL (`$where`, `$ne`) antes de procesar filtros. DTOs inmutables con `readonly`. | `security.util.ts`<br>`sql-mock.service.ts`<br>`ticket.model.ts` |

```
// Extracto de security.util.ts — Sanitización defensiva pura contra inyecciones SQL / NoSQL
export function sanitizeQueryInput(input: string | null | undefined): string {
  if (!input || typeof input !== 'string') return '';
  let sanitized = input.trim();
  for (const pattern of SQL_INJECTION_PATTERNS) {
    sanitized = sanitized.replace(pattern, '');
  }
  return sanitized.replace(/\s{2,}/g, ' ');
}
```

**Evidencia 03 — Consola Bento NOC (/dashboard) para Rol ADMIN con Cockpit de Telemetría** `canActivate: [authGuard, roleGuard(['ADMIN'])]`



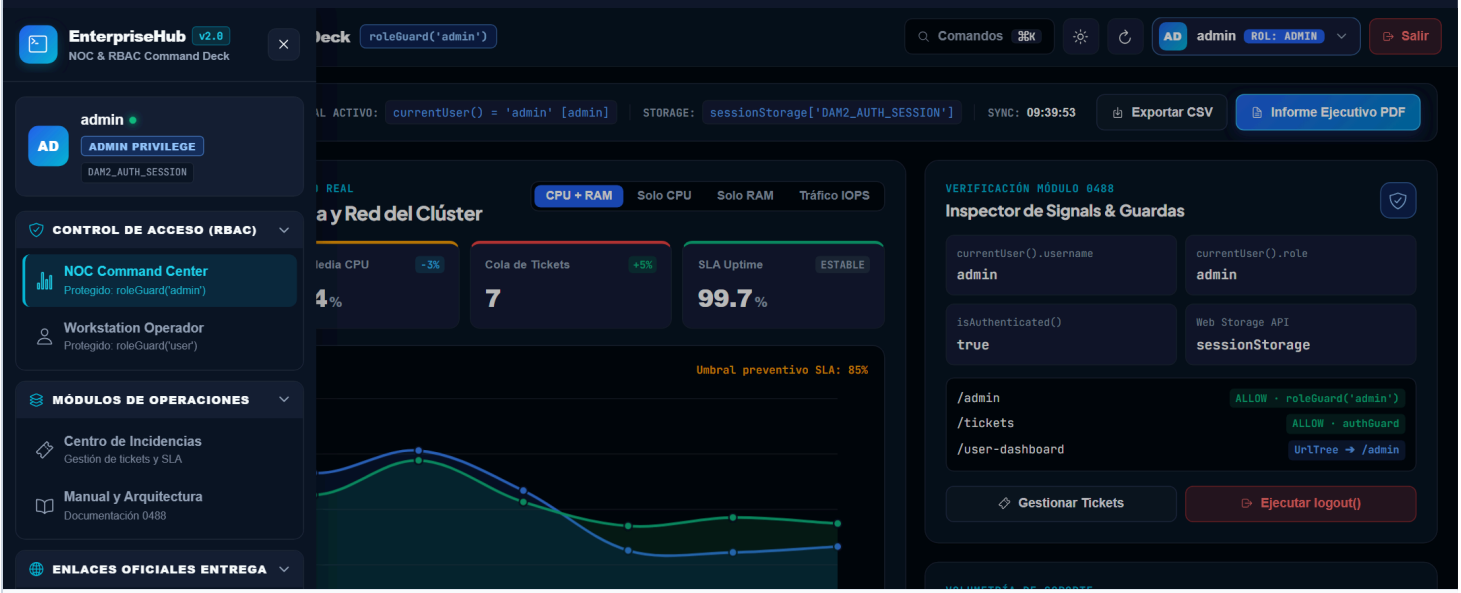

**Análisis:** Vista protegida de administración con inspector reactivo de Signals, matriz de privilegios y bastidor Blade con diagnóstico ICMP.

## 4. Pipeline de Functional Guards, Interceptors y Directiva *hasRole

```
// role.guard.ts — Factoría Funcional de Control de Acceso Granular (RBAC)
export const roleGuard = (allowedRoles: SystemRole[] | SystemRole | string[]): CanActivateFn => {
  return (_route, state) => {
    const authService = inject(AuthService);
    const router = inject(Router);
    if (!authService.isAuthenticated()) {
      return router.createUrlTree(['/login'], { queryParams: { returnUrl: state.url } });
    }
    const session = authService.currentUser();
    const userRole = (session?.role ?? '').toUpperCase();
    const normalizedAllowed = Array.isArray(allowedRoles) ? allowedRoles.map(r => String(r).toUpperCase()) :
[String(allowedRoles).toUpperCase()];
    if (normalizedAllowed.includes(userRole)) {
      authService.recordAudit('GUARD', 'SUCCESS', `roleGuard: Acceso AUTORIZADO a '${state.url}' para
'${session?.username}'`);
      return true;
    }
    authService.recordAudit('GUARD', 'WARN', `roleGuard: Acceso DENEGADO a '${state.url}' para rol
'${userRole}'`);
    return router.createUrlTree([userRole === 'USER' ? '/user-dashboard' : '/tickets']);
  };
};
```

**Evidencia 04 — Menú Lateral con Renderizado Condicional Mediante Directiva *hasRole="['ADMIN']"**

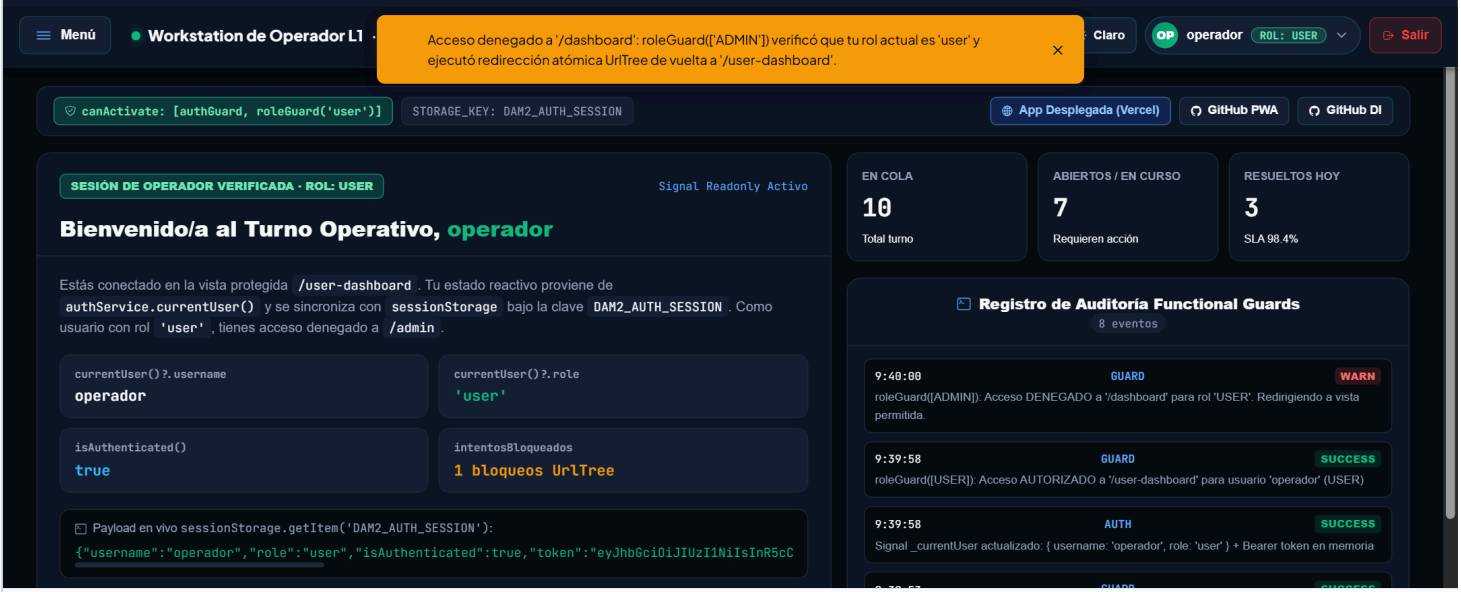
Directiva Estructural Standalone




**Análisis:** La directiva estructural `*hasRole="['ADMIN']"` retira físicamente del DOM los enlaces privilegiados cuando el usuario conectado no ostenta el rol de administración.

## Evidencia 05 — Workstation de Operador (/user-dashboard) y Bloqueo Atómico roleGuard(['ADMIN'])

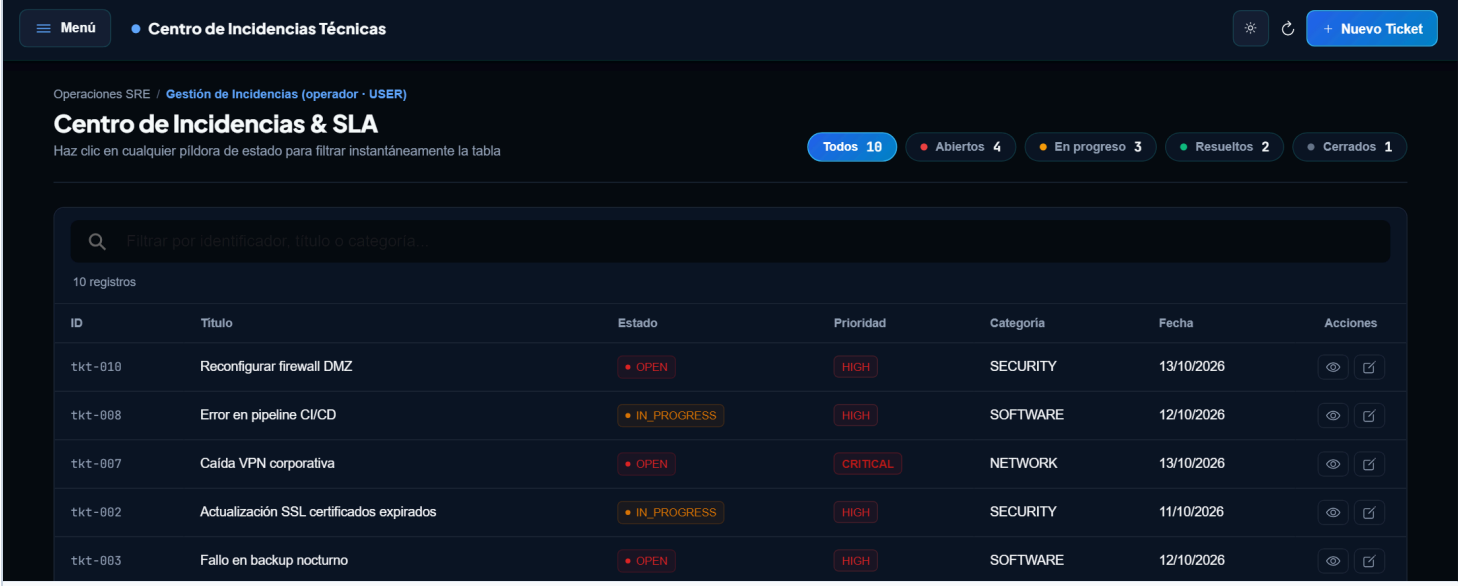
`Sesión: operador · Rol: USER`




**Análisis:** Al pulsar "Probar acceso a /dashboard", la factoría `roleGuard(['ADMIN'])` intercepta la petición del operador y ejecuta una redirección atómica `UrlTree` de vuelta a `/user-dashboard` con registro en consola SRE.

## Evidencia 06 — Centro de Incidencias (/tickets) con Skeleton Loaders y Filtros por SLA

```
canActivate: [authGuard,
roleGuard(['ADMIN','SUPERVISOR','USER'])]
```




**Análisis:** Interfaz de gestión de incidencias con tabla desacoplada OnPush, filtrado instantáneo por severidad técnica y sanitización previa a la persistencia.